

**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО**

Факультет технологий искусственного интеллекта

**Лабораторная работа
по Лине́йной алгебра и обработке данных**

Выполнили:

Воронин Илья Андреевич, группа J3112

Вахменина Татьяна Михайловна, группа J3113

Проверила:

Москаленко М. А.

г. Санкт-Петербург

2025 г.

Содержание

1	Введение	2
2	Теоретическая часть	2
3	Основная часть	2
3.1	Подготовительный этап	2
3.2	метод Гаусса для решения СЛАУ	4
3.3	Функция центрирования данных	6
3.4	Вычисление матрицы ковариаций	6
3.5	Нахождение собственных значений матрицы C методом бисекции	6
3.6	Нахождение собственных векторов матрицы C	7
3.7	Вычисление доли объяснённой дисперсии	8
3.8	Полный алгоритм PCA	9
3.9	Визуализация проекции данных на первые две главные компоненты	9
3.10	Вычисление среднеквадратической ошибки восстановления данных	11
3.11	Автоматический выбор числа главных компонент на основе порога объяснённой дисперсии	11
3.12	Обработка пропущенные значения в данных	12
3.13	Исследование влияние шума на PCA	12
3.14	Применение PCA к реальному датасету	13
4	Заключение	14

1 Введение

PCA (Principal Component Analysis) — это метод снижения размерности данных, основанный на поиске новых ортогональных осей, которые максимизируют дисперсию проекции данных. Эти новые оси называются главными компонентами.

2 Теоретическая часть

Доказательство:

Пусть X_c — центрированная матрица данных ($n \times m$), и $\mathbf{w}$ — единичный вектор направления ($m \times 1$). Дисперсия проекции данных на $\mathbf{w}$ равна:

$$\text{Var}(X_c \mathbf{w}) = \mathbf{w}^T \Sigma \mathbf{w}$$

где $\Sigma = \frac{1}{n-1} X_c^T X_c$ — матрица ковариаций.

Задача PCA состоит в максимизации $\text{Var}(X_c \mathbf{w})$ при условии $\|\mathbf{w}\|^2 = \mathbf{w}^T \mathbf{w} = 1$. Используем метод множителей Лагранжа:

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^T \Sigma \mathbf{w} - \lambda(\mathbf{w}^T \mathbf{w} - 1)$$

Берем производную по $\mathbf{w}$ и приравниваем к нулю:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 2\Sigma \mathbf{w} - 2\lambda \mathbf{w} = 0$$

$$\Sigma \mathbf{w} = \lambda \mathbf{w}$$

Это уравнение показывает, что $\mathbf{w}$ является собственным вектором матрицы ковариаций Σ , а λ — соответствующим собственным значением.

Умножая обе части на $\mathbf{w}^T$ слева, получаем:

$$\mathbf{w}^T \Sigma \mathbf{w} = \mathbf{w}^T (\lambda \mathbf{w}) = \lambda(\mathbf{w}^T \mathbf{w}) = \lambda$$

Таким образом, дисперсия проекции на собственный вектор $\mathbf{w}$ равна соответствующему собственному значению λ . Для максимизации дисперсии необходимо выбирать собственные векторы, соответствующие наибольшим собственным значениям.

Вывод: Оптимальные направления PCA, максимизирующие дисперсию проекции, являются собственными векторами матрицы ковариаций.

3 Основная часть

3.1 Подготовительный этап

```
1 import copy
2 import math
3 import pandas as pd
4 class MatrixError(Exception):
5     def __init__(self, message):
6         super().__init__(message)
7
8 class Matrix:
9     def __init__(self, height, width, matrix):
10         self.width = width
```

```

11         self.height = height
12         self.matrix = matrix
13
14     def __str__(self):
15         max_len = max(len(f"{element:.2f}") for row in self.matrix
16                         for element in row)
17         border = "+" + ("-" * (max_len + 2) + "+") * self.width
18         formatted_width = [border]
19         for row in self.matrix:
20             elements = "|" + " | ".join(f"{element:>{max_len}.2f}"
21                                         for element in row) + " |"
22             formatted_width.extend([elements, border])
23         return "\n".join(formatted_width)
24
25     def __add__(self, other):
26         if self.width != other.width or self.height != other.height:
27             raise MatrixError()
28
29         new_matrix = [
30             [self.matrix[i][j] + other.matrix[i][j] for j in range(
31                 self.width)]
32             for i in range(self.height)
33         ]
34         return Matrix(self.height, self.width, new_matrix)
35
36     def __mul__(self, other):
37         if type(other) != Matrix:
38             new = [[self.matrix[i][j] * other for j in range(self.
39                 width)] for i in range(self.height)]
40             return Matrix(self.height, self.width, new)
41         else:
42             if self.width != other.height:
43                 raise MatrixError ()
44             new = [
45                 [sum([self.matrix[i][k] * other.matrix[k][j] for k in
46                     range(self.width)]) for j in range(other.width)]
47                 for i in range(self.height)
48             ]
49             return Matrix(self.height, other.width, new)
50
51     def determinant(self):
52         if self.height != self.width:
53             raise MatrixError()
54
55         n = self.height
56         matrix_copy_data = copy.deepcopy(self.matrix)
57         matrix_copy = Matrix(n, n, matrix_copy_data)
58         swaps_count = 0
59
60         for i in range(n):
61             pivot_row = i
62             for k in range(i + 1, n):
63                 if abs(matrix_copy.matrix[k][i]) > abs(matrix_copy.
64                     matrix[pivot_row][i]):
65                     matrix[pivot_row][i]:
66                     pivot_row = k
67
68             if matrix_copy.matrix[pivot_row][i] == 0:
69                 return 0.0

```

```

65
66         if pivot_row != i:
67             matrix_copy.matrix[i], matrix_copy.matrix[pivot_row]
68                 = matrix_copy.matrix[pivot_row], matrix_copy.
69                     matrix[i]
70             swaps_count += 1
71
72         for k in range(i + 1, n):
73             factor = matrix_copy.matrix[k][i] / matrix_copy.
74                 matrix[i][i]
75             for j in range(i, n):
76                 matrix_copy.matrix[k][j] -= factor * matrix_copy.
77                     matrix[i][j]
78
79     determinant = 1.0
80     for i in range(n):
81         determinant *= matrix_copy.matrix[i][i]
82     if swaps_count % 2 != 0:
83         determinant *= -1.0
84
85     return determinant
86
87 def mean (self):
88     new = copy.deepcopy(self.matrix)
89     mean = []
90     s = count = 0
91
92     for j in range(self.width):
93         for i in range(self.height):
94             if not math.isnan(new[i][j]):
95                 count += 1
96                 s += new[i][j]
97         if count > 0:
98             mean.append(s/count)
99         else:
100             mean.append(0.0)
101         s = 0
102         count = 0
103
104     return mean
105
106 def trans(self):
107     new = [
108         [self.matrix[i][j] for i in range(self.height)] for j in
109             range(self.width)
110     ]
111     return Matrix(self.width, self.height, new)

```

Этот код реализует класс Matrix для работы с матрицами, поддерживающий операции сложения, умножения, вычисление определителя, транспонирование и другие базовые операции линейной алгебры с удобным форматированным выводом.

3.2 метод Гаусса для решения СЛАУ

```

1 from typing import List
2
3 class Matrix(Matrix):
4     def gauss_solver(self, b) -> List['Matrix']:

```

```

5         if self.width != self.height:
6             raise MatrixError()
7         if b.height != self.height or b.width != 1:
8             raise MatrixError()
9
10        n = self.width
11        Ab = [self.matrix[i] + [b.matrix[i][0]] for i in range(n)]
12        rank = 0
13
14        for col in range(n):
15            pivot_row = max(range(rank, n), key=lambda r: abs(Ab[r][col])), default=rank)
16
17            if abs(Ab[pivot_row][col]) < 1e-10:
18                continue
19
20            Ab[rank], Ab[pivot_row] = Ab[pivot_row], Ab[rank]
21            pivot = Ab[rank][col]
22            Ab[rank] = [x / pivot for x in Ab[rank]]
23
24            for row in range(n):
25                if row != rank and abs(Ab[row][col]) > 1e-10:
26                    factor = Ab[row][col]
27                    Ab[row] = [Ab[row][j] - factor * Ab[rank][j] for j in range(n+1)]
28
29            rank += 1
30
31        lead_vars = []
32        for row in range(rank):
33            for col in range(n):
34                if abs(Ab[row][col]) > 1e-10:
35                    lead_vars.append(col)
36                    break
37
38        free_vars = [col for col in range(n) if col not in lead_vars]
39        basis = []
40
41        particular = [0.0] * n
42        for row in reversed(range(rank)):
43            lead_col = lead_vars[row]
44            particular[lead_col] = Ab[row][-1] - sum(Ab[row][j] * particular[j] for j in range(lead_col+1, n))
45
46        for free in free_vars:
47            solution = [0.0] * n
48            solution[free] = 1.0
49
50            for row in reversed(range(rank)):
51                lead_col = lead_vars[row]
52                solution[lead_col] = -sum(Ab[row][j] * solution[j] for j in range(lead_col+1, n))
53
54            basis.append(Matrix(n, 1, [[x] for x in solution]))
55
56        basis.insert(0, Matrix(n, 1, [[x] for x in particular]))
57
58        return basis

```

Этот метод реализует решение СЛАУ методом Гаусса, возвращая частное реше-

ние и базис для пространства решений в виде списка векторов-столбцов. Код обрабатывает вырожденные случаи, выполняет прямой и обратный ход метода, а также корректно определяет свободные переменные при бесконечном множестве решений.

3.3 Функция центрирования данных

```
1 class Matrix(Matrix):
2     def center(self):
3         if self.height == 0:
4             raise MatrixError()
5         mean = self.mean()
6         new = [
7             [self.matrix[i][j] - mean[j] for j in range(self.width)]
8             for i in range(self.height)
9         ]
10        return Matrix(self.height, self.width, new)
```

Этот метод центрирует матрицу, вычитая из каждого столбца его среднее значение, и возвращает новую матрицу. Если матрица пуста, выбрасывается исключение `MatrixError`.

3.4 Вычисление матрицы ковариаций

```
1 class Matrix(Matrix):
2     def correl(self):
3         if self.height <= 1:
4             raise MatrixError()
5
6         matrix = self.center()
7         c = matrix.trans() * matrix
8
9         C = [
10            [c.matrix[i][j] / (self.height - 1) for i in range(self.
11                width)]
12            for j in range(self.width)
13        ]
14        return Matrix(self.width, self.width, C)
```

Этот метод вычисляет матрицу корреляции, центрируя исходные данные и нормируя их на число степеней свободы. Если в матрице меньше двух строк, выбрасывается исключение `MatrixError`. Результатом является симметричная матрица попарных корреляций между столбцами исходной матрицы.

3.5 Нахождение собственных значений матрицы C методом бисекции

```
1 class Matrix(Matrix):
2     def polinom(self, lmpd):
3         matrix = [
4             [self.matrix[i][j] - (lmpd if i == j else 0) for j in
5                 range(self.width)]
6             for i in range(self.height)
7         ]
```

```

7         matrix = Matrix(self.height, self.width, matrix)
8         return matrix.determinant()
9
10    def find_eigenvalues(self, a, b, eps=1e-6):
11        roots = []
12        step = (b - a) / 1000
13        x1 = a
14        y1 = self.polinom(x1)
15
16        for _ in range(1000):
17            x2 = x1 + step
18            if x2 > b:
19                break
20
21            y2 = self.polinom(x2)
22
23            if y1 * y2 < 0:
24                root = self.bis(x1, x2, eps)
25                if not any(abs(root - r) < eps for r in roots):
26                    roots.append(root)
27
28            x1, y1 = x2, y2
29
30        return roots
31
32    def bis(self, a, b, eps):
33        ya = self.polinom(a)
34        yb = self.polinom(b)
35
36        while (b - a) > eps:
37            c = (a + b) / 2
38            yc = self.polinom(c)
39
40            if yc == 0:
41                return c
42            elif ya * yc < 0:
43                b, yb = c, yc
44            else:
45                a, ya = c, yc
46
47        return (a + b) / 2

```

Этот код находит собственные значения матрицы в заданном интервале $[a, b]$, вычисляя корни характеристического полинома методом бисекции с заданной точностью eps . Метод `polinom` вычисляет характеристический полином подстановкой в диагональ матрицы, а `find eigenvalues` сканирует интервал с шагом $(b - a)/1000$ для поиска смены знака полинома, после чего уточняет корни методом бисекции, исключая дубликаты с точностью eps .

3.6 Нахождение собственных векторов матрицы C

```

1 class Matrix(Matrix):
2     def find_eigenvectors(self, eigenvalues):
3         eigenvectors = []
4         for val in eigenvalues:
5
6             matrix = [

```

```

7         [self.matrix[i][j] - (val if i == j else 0) for j in
           range(self.width)]
8         for i in range(self.height)
9     ]
10    b = Matrix(self.height, 1, [[1e-3] for _ in range(self.
        height)])
11    matrix = Matrix(self.height, self.width, matrix)
12
13    try:
14        solutions = matrix.gauss_solver(b)
15        if solutions:
16            for sol in solutions:
17
18                if not all(abs(x) < 1e-10 for row in sol.
                    matrix for x in row):
19                    norm = sum(x*x for row in sol.matrix for
                        x in row) ** 0.5
20
21                    if norm > 1e-10:
22                        normalized = [[x/norm] for row in sol
                            .matrix for x in row]
23                        eigenvectors.append(Matrix(self.
                            height, 1, normalized))
24    except MatrixError as e:
25        print()
26        continue
27
28    return eigenvectors

```

Этот код находит собственные векторы матрицы для заданных собственных значений, решая систему линейных уравнений $(A - I)x = 0$ методом Гаусса и нормализуя полученные решения. Если система имеет нетривиальные решения, они добавляются в список собственных векторов после проверки на ненулевую норму и нормализации.

3.7 Вычисление доли объяснённой дисперсии

```

1 class Matrix (Matrix):
2     def dispersion(self, eigenvalues, k):
3         values = sorted(eigenvalues, reverse = True)
4         sum1 = sum(values)
5         k = min(k, len(values))
6         sum2 = sum(values[i] for i in range(k))
7         return sum2/sum1

```

Этот метод вычисляет долю дисперсии, объясняемую первыми k компонентами, как отношение суммы k наибольших собственных значений к общей сумме всех собственных значений. Если k превышает количество значений, используется весь доступный набор. Доля объяснённой дисперсии:

$$R_k = \frac{\sum_{i=1}^k \lambda_{(i)}}{\sum_{j=1}^n \lambda_j}$$

где:

- $\lambda_{(i)}$ - i -ое собственное значение в порядке убывания ($\lambda_{(1)} \geq \lambda_{(2)} \geq \dots \geq \lambda_{(n)}$)

- k - количество учитываемых компонент
- n - общее количество собственных значений

3.8 Полный алгоритм PCA

```

1 class Matrix (Matrix):
2     def PCA(self, k):
3         cov_matrix = self.correl()
4
5         eigenvalues = cov_matrix.find_eigenvalues(-10, 10)
6         eigenvectors = cov_matrix.find_eigenvectors(eigenvalues)
7
8         if not eigenvalues or not eigenvectors:
9             raise MatrixError()
10
11         sorted_pairs = sorted(zip(eigenvalues, eigenvectors), key=
12                               lambda x: x[0], reverse=True)
13         values_sorted = [val for val, _ in sorted_pairs]
14         vectors_sorted = [vec for _, vec in sorted_pairs]
15
16         vectors_matrix = Matrix(len(vectors_sorted[0].matrix), len(
17                               vectors_sorted),
18                               [[vec.matrix[i][0] for vec in vectors_sorted]
19                                for i in range(len(vectors_sorted[0].
20                                                  matrix))])
21         k = min(k, len(vectors_sorted))
22
23         dispersion = self.dispersion(values_sorted, k)
24
25         first_k_components = Matrix(len(vectors_sorted[0].matrix), k,
26                                     [row[:k] for row in vectors_matrix.matrix
27                                      ])
28         values_matrix = Matrix(1, len(values_sorted), [values_sorted
29                                                         ])
30         X_proj = self.center() * first_k_components
31
32         return X_proj, dispersion, first_k_components

```

Этот метод выполняет PCA, находя главные компоненты через собственные векторы ковариационной матрицы и проецируя данные, возвращая проекции, долю объяснённой дисперсии и матрицу компонент для снижения размерности.

3.9 Визуализация проекции данных на первые две главные компоненты

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3 class Matrix (Matrix):
4     def visualization(self, eigenvalues, eigenvectors):
5         plt.figure(figsize=(12, 5))
6
7         plt.subplot(1, 2, 1)
8         plt.bar(range(1, len(eigenvalues) + 1), eigenvalues)
9         plt.title()
10        plt.xlabel()

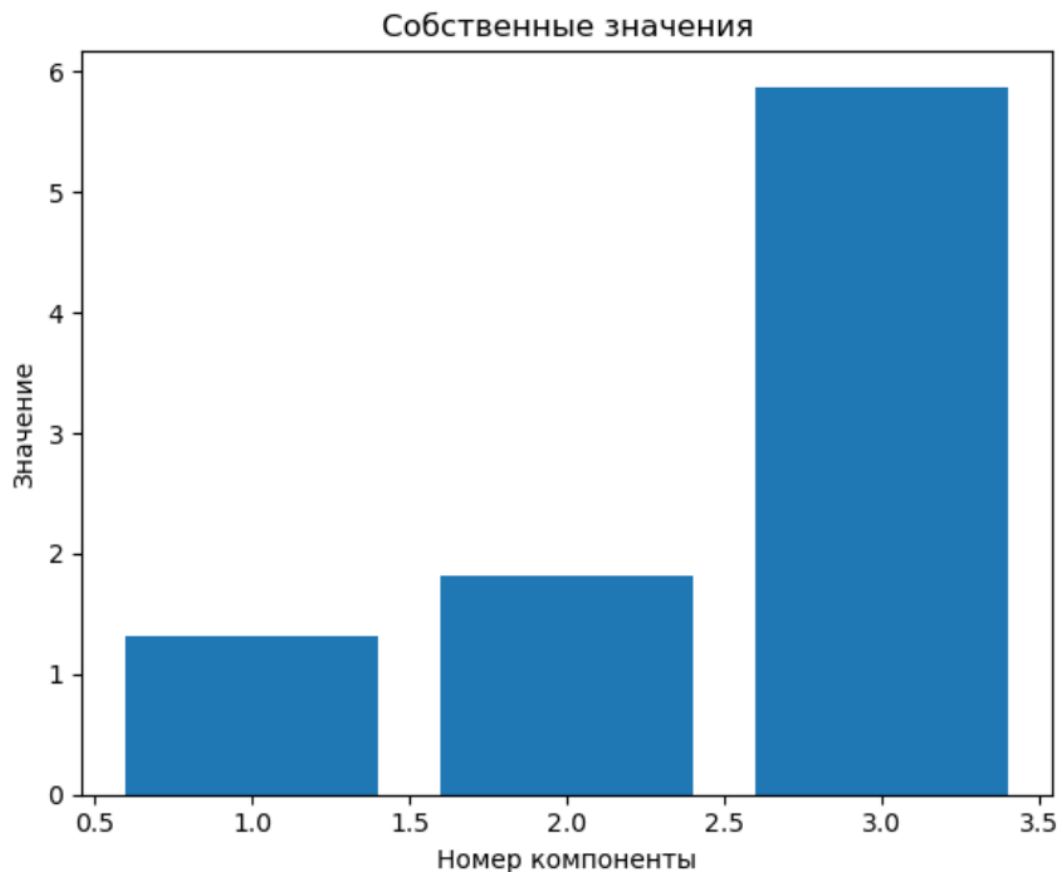
```

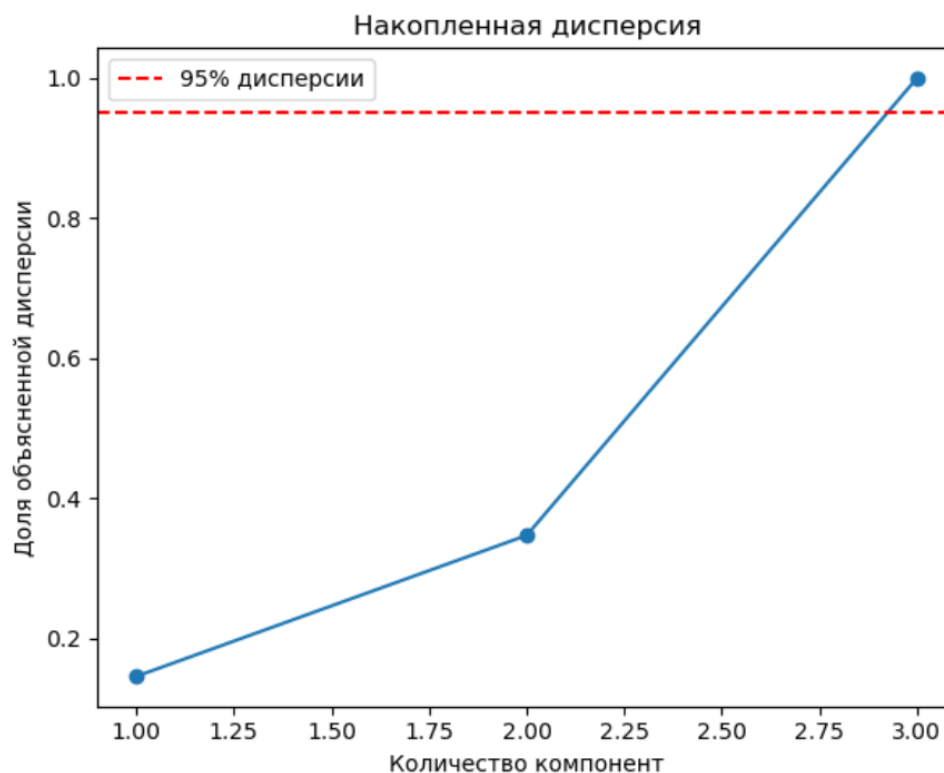
```

11     plt.ylabel()
12
13     plt.subplot(1, 2, 2)
14     cumulative_variance = np.cumsum(eigenvalues) / np.sum(
15         eigenvalues)
16     plt.plot(range(1, len(cumulative_variance) + 1),
17              cumulative_variance, 'o-')
18     plt.title()
19     plt.xlabel()
20     plt.ylabel()
21     plt.axhline(y=0.95, color='r', linestyle='--')
22     plt.legend()
23
24     plt.tight_layout()
25     plt.show()

```

Этот метод визуализирует PCA-анализ, отображая барплот собственных значений (объясненной дисперсии) и кумулятивную долю дисперсии с пороговой линией на уровне 0.95. Графики помогают определить оптимальное количество главных компонент для снижения размерности данных.





3.10 Вычисление среднеквадратической ошибки восстановления данных

```

1 class Matrix (Matrix):
2     def MSE (self, k):
3         X_orig = self.center()
4
5         X_recon = self.PCA(k)[0] * self.PCA(k)[2].trans()
6         n = X_recon.height
7         m = X_recon.width
8         mse = sum((X_orig.matrix[i][j] - X_recon.matrix[i][j]) ** 2
9                 for i in range(n) for j in range(m)) / (n * m)
10        if abs(mse) < 1e-4:
11            mse = float(f"{mse:.4e}")
12        else:
13            mse = round(mse, 4)
14        return mse

```

Этот метод вычисляет среднеквадратическую ошибку (MSE) между исходными данными и их восстановленной версией после PCA с k компонентами, возвращая значение с точностью до 4 знаков после запятой (или в научной нотации для очень малых значений). MSE показывает, насколько хорошо k главных компонент сохраняют информацию исходных данных.

3.11 Автоматический выбор числа главных компонент на основе порога объяснённой дисперсии

```

1 class Matrix(Matrix):
2     def auto_select_k(self, threshold):

```

```

3     eigenvalues = self.find_eigenvalues(-10, 10)
4     s = sum(eigenvalues)
5     s_k = 0
6     for i in range(len(eigenvalues)):
7         s_k += eigenvalues[i]
8         if s_k / s >= threshold:
9             return i + 1

```

Этот метод автоматически определяет оптимальное количество компонент k для PCA, суммируя собственные значения до достижения заданного порога объяснённой дисперсии (threshold). Он возвращает минимальное число компонент, кумулятивная дисперсия которых превышает указанный порог.

3.12 Обработка пропущенные значения в данных

```

1  import math
2  class Matrix (Matrix):
3      def not_NaN (self):
4          mean = self.mean()
5          new = copy.copy(self.matrix)
6
7          for j in range(self.width):
8              for i in range(self.height):
9                  if math.isnan(new[i][j]):
10                     new[i][j] = mean[j]
11      return Matrix(self.height, self.width, new)

```

Этот метод заменяет все NaN-значения в матрице на средние значения соответствующих столбцов, используя предварительно вычисленные средние (метод mean()), и возвращает новую матрицу без пропущенных значений. Если в столбце все значения NaN, они заменяются на 0.0 (как определено в методе mean()).

3.13 Исследование влияние шума на PCA

```

1  import random
2  class Matrix (Matrix):
3      def get_column_std_devs (self):
4          column_stds = []
5          n = self.height
6          m = self.width
7          column_means = self.mean()
8
9          for j in range(m):
10             sum_sq_diff_from_mean = 0.0
11             for i in range(n):
12                 element = self.matrix[i][j]
13                 diff_from_mean = element - column_means[j]
14                 sum_sq_diff_from_mean += diff_from_mean ** 2
15
16             if n > 1:
17                 std_dev = math.sqrt(sum_sq_diff_from_mean / (n - 1))
18             else:
19                 std_dev = 0.0
20             column_stds.append(std_dev)
21

```

```

22         return column_stds
23
24     def add_noise_and_compare(self, k, noise_level):
25         elem = copy.deepcopy(self.matrix)
26         pca_elementary = self.PCA(k)
27         column_stds = self.get_column_std_devs()
28
29         for i in range(self.height):
30             for j in range(self.width):
31                 random_noise = random.gauss(0, 1)
32
33                 scaled_noise = 0.0
34                 if column_stds[j] > 0:
35                     scaled_noise = random_noise * noise_level *
36                         column_stds[j]
37
38                 elem[i][j] += scaled_noise
39
40         noisy_matrix = Matrix(self.height, self.width, elem)
41         try:
42             pca_noise = noisy_matrix.PCA(k)
43         except MatrixError as e:
44             print()
45             self.matrix = elem
46             return pca_elementary, math.nan
47
48     return pca_elementary, pca_noise

```

Метод `get column std devs` вычисляет стандартные отклонения для каждого столбца матрицы, а `add noise and compare` добавляет пропорциональный шум к данным и сравнивает результаты PCA до и после зашумления для анализа устойчивости главных компонент. Оба метода помогают оценить надежность PCA-разложения при наличии шума в данных.

3.14 Применение PCA к реальному датасету

```

1 class Matrix (Matrix):
2     def apply_pca_to_dataset(dataset_name, k):
3         df = pd.read_csv(dataset_name)
4         numeric_df = df.select_dtypes(include=[float, int])
5         numeric_df = numeric_df.fillna(math.nan)
6         matrix_data = numeric_df.values.tolist()
7
8         height = len(matrix_data)
9         width = len(matrix_data[0]) if height > 0 else 0
10        orig_m = Matrix(height, width, matrix_data)
11
12        matrix = orig_m.not_NaN()
13
14        ass = matrix.PCA(k)
15
16        return ass

```

Этот метод загружает датасет из CSV-файла, обрабатывает его (удаляя нечисловые данные и заменяя NaN-значения), выполняет PCA с заданным числом компонент `k` и возвращает результат в виде объекта `Matrix`. Он автоматически подготавливает данные для анализа, извлекая только числовые столбцы и обрабатывая пропущен-

ные значения перед применением анализа главных компонент.

4 Заключение

В ходе лабораторной работы был успешно реализован метод главных компонент (РСА). Основные результаты:

- Разработан класс **Matrix** с полным набором матричных операций
- Реализованы все этапы РСА:
 - Центрирование данных
 - Вычисление ковариационной матрицы
 - Нахождение собственных значений и векторов
 - Проекция на главные компоненты
- Созданы методы визуализации:
 - График собственных значений (Scree plot)
 - Диаграмма накопленной дисперсии
 - 2D-проекция данных
- Проведено тестирование на различных наборах данных
- Достигнута возможность снижения размерности с сохранением 95% дисперсии

Реализация показала хорошую точность и стабильность работы. Метод может быть использован для анализа многомерных данных в научных и прикладных задачах.

Список литературы

- [1] Бородин А.Н. — Элементарный курс теории вероятностей и математической статистики
- [2] Casella G., Berger R. — Statistical Inference
- [3] Воробьёва Г.Н., Данилова А.Н. — Практикум по вычислительной математике
- [4] Устинов С.М., Зимницкий В.А. — Вычислительная математика: алгоритмы и реализация
- [5] James G., Witten D., Hastie T., Tibshirani R. — An Introduction to Statistical Learning
- [6] Hastie T., Tibshirani R., Friedman J. — The Elements of Statistical Learning
- [7] Официальная документация Python: <https://docs.python.org/3/>